EXPRESSÕES REGULARES

*O servidor estava sob força bruta e o analista rodou `grep 10.0.0.1 /var/log/auth.log`. Vieram as linhas certas — e uma com `id=10a0b0c1`, que não tem endereço nenhum. O ponto que ele digitou querendo dizer **ponto** significa, para a ferramenta, **qualquer caractere**. Na tentativa seguinte, `grep 'Failed+'` devolveu zero linhas — sem erro, sem aviso: o `grep` sem `-E` não conhece o `+`. Um padrão descreve **forma**, e há **três dialetos** dessa notação na mesma máquina.*

064
GUIAS LINUX

ANATOMIA DE UM PADRÃO REAL — ENDEREÇO IPV4

`^` `(` `[0-9]` `{1,3}` `\.` `)` `{3}` `[0-9]` `{1,3}` `$`

`^ $` âncora
Condição sobre a **posição**: não consome caractere. Daí `^$` ser "linha vazia".

`( )` grupo
Delimita o item para o quantificador **e captura** o trecho como `\1`.

`[ ]` classe
Um caractere dentre os listados. `[^0-9]` nega; `[.]` desarma o ponto.

`{n,m}` quantifica
Repete só o item **imediatamente anterior**. Em `ab*`, o `*` governa só o `b`.

`\.` literal
A barra **tira** o poder do símbolo. `\.` é ponto de verdade.

Lido da esquerda para a direita: *da borda esquerda da linha, três repetições de "um a três dígitos seguidos de ponto", depois mais um a três dígitos, e a borda direita.* Note o que ele **não** diz: nada impede `999.999.999.999`. **Expressão regular reconhece forma; validar valor é outro problema** — trabalho de `awk` ou de uma linguagem. Quando o padrão passa de três linhas ou precisa *contar*, troque de ferramenta: HTML, JSON e parênteses balanceados **não são linguagens regulares** e nenhum padrão resolve o caso geral. E não confunda com o **globbing**: o glob é do **shell** e opera sobre nomes de arquivo (`*` = qualquer sequência); a expressão regular é da **ferramenta** e opera sobre linhas (`*` = zero ou mais do item anterior).

OS TRÊS SABORES QUE CONVIVEM NA MESMA MÁQUINA

Construto	BRE grep · sed · expr	ERE grep -E · sed -E · awk	PCRE grep -P · perl · python
uma ou mais	`a\+`	`a+`	`a+`
opcional	`a\?`	`a?`	`a?`
repetição contada	`a\{2,5\}`	`a{2,5}`	`a{2,5}`
grupo	`\(ab\)`	`(ab)`	`(ab)`
alternância	`a|b`	`a|b`	`a|b`
dígito	`[[:digit:]]`	`[[:digit:]]`	`\d`
preguiçoso · olhar adiante	não existe	não existe	`.*?` `(?=…)`

A regra que resolve quase tudo: em **BRE** o construto avançado exige barra invertida e o símbolo cru é literal; em **ERE** é o contrário — a barra só serve para *tirar* o poder. **Na dúvida,** `grep -E`: é o mesmo padrão que você escreveria em `awk`, `sed -E` ou em qualquer linguagem moderna. O `egrep` e o `fgrep` ainda rodam, mas imprimem aviso de depreciação no Kali atual. E há uma quarta variante que não é expressão regular nenhuma: `grep -F` desliga a interpretação inteira, busca texto literal e é a mais rápida.

`grep ^root /etc/passwd`
Padrão vai entre aspas simples, sempre. Sem elas o shell processa `*`, `?`, `[…]`, `$` e `\` **antes** da ferramenta ver o padrão — e o que chega ao `grep` já não é o que você escreveu.

`grep -E '^(erro|falha)'`
A alternância tem a menor precedência de todas. `^erro|falha` significa "começa com erro" **ou** "contém falha" em qualquer posição — quase nunca o que se quer. Agrupe.

O `.*` é guloso e atravessa delimitador. Numa linha com três campos entre aspas, `grep -oE '".*"'` devolve **uma cadeia gigante**, da primeira à última aspa. Negue em vez de generalizar: **`"[^"]*"`** — em PCRE existe também o preguiçoso **`".*?"`**. **Faixa depende da localidade:** `[a-z]` muda de resultado conforme `LC_ALL`; as **classes POSIX nomeadas** são estáveis — `[[:digit:]]`, `[[:alpha:]]`, `[[:alnum:]]`, `[[:space:]]`, `[[:upper:]]`, `[[:punct:]]`, `[[:xdigit:]]`. Dentro dos colchetes as regras mudam: o `^` só nega na **primeira** posição, o `]` só é literal **logo no início** e o `-` só é literal no **começo ou no fim**. Bordas de palavra `\<pai\>` e `\bpai\b` impedem que `10.0.0.1` case dentro de `10.0.0.14`.

Retrocesso catastrófico (ReDoS) — não rode em produção. O motor POSIX (`grep -E`, `sed`, `awk`) usa autômato finito: casa em tempo proporcional ao tamanho do texto e **não retrocede**. O motor PCRE (`grep -P`, linguagens) retrocede — e um quantificador aninhado explode: `echo "$(printf 'a%.0s' {1..40})" | grep -qP '(a+)+b'` prende um núcleo por minutos, porque o motor tenta um número **exponencial** de divisões antes de desistir. É por isso que `(a|ab)` casa `ab` em POSIX (mais longo) e `a` em PCRE (primeira alternativa que fecha). Defesas: prefira `-E` a `-P` quando os dois servirem; em PCRE, nada de quantificador dentro de grupo quantificado, e classe negada no lugar de `.*`.

RESUMO RÁPIDO

- ✓ **Padrão descreve forma, não texto literal.** O `.` é qualquer caractere: `grep 10.0.0.1` casa com `10a0b0c1`. Ponto de verdade se escreve `\.`.
- ✓ **Padrão vai entre aspas simples, sempre** — senão o shell consome `*`, `?`, `$`, `[` e `\` antes da ferramenta.
- ✓ **São três dialetos.** BRE exige `\+ \? \{ \( \|`; ERE usa os símbolos direto; PCRE acrescenta `\d`, `\b`, `.*?` e olhar adiante. **Na dúvida, ERE.**
- ✓ **Classe POSIX nomeada é mais estável que faixa:** `[[:digit:]]` não muda com `LC_ALL`; `[0-9]` e `[a-z]` mudam.
- ✓ **Quantificador governa só o item anterior** — para repetir mais de um caractere, agrupe. E a alternância tem a **menor** precedência: `^a|b` quase nunca é o que se quer.
- ✓ **`.*` é guloso e atravessa delimitadores.** Prefira a classe negada `[^"]*`.
- ✓ **POSIX não retrocede; PCRE sim.** Daí `(a|ab)` casar diferente e o ReDoS existir só no segundo.
- ✓ **Nem tudo é linguagem regular.** HTML, JSON e aninhamento não se resolvem com padrão: troque para `awk`, `jq` ou Python.

NO KALI

`grep -F -- "$ALVO"` para tudo que veio de fora. Filtrar a saída de uma varredura por um domínio guardado em variável trata o **conteúdo** da variável como padrão: um ponto vira coringa e um `[` mal fechado derruba o comando com *Unmatched [*. Se o alvo veio de arquivo de escopo, de argumento ou de resposta de API, use `-F`; o `--` fecha a lista de opções e evita que um alvo começando com traço seja lido como *flag*. **Wordlist se filtra com ERE, e o ganho é de horas:** `grep -E '^.{8,}$' rockyou.txt | grep -E '[[:upper:]]' | grep -E '[[:digit:]]'` reduz a lista à política de senha do alvo antes de entregá-la à ferramenta — três `grep` em cadeia são mais legíveis, e em geral mais rápidos, que um padrão único com olhar adiante. **Extrair indicador de log é o uso diário**, e quem faz isso é o `-o`: `grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'` puxa IPv4, `grep -oE '[0-9a-f]{32}'` puxa *hashes* MD5 e `grep -oPi 'href="\K[^"]+'` puxa URLs de um HTML salvo — o `\K` do PCRE descarta o que veio antes e dispensa o grupo de captura. **ReDoS é classe de vulnerabilidade, não incômodo:** uma aplicação que valida entrada do usuário com quantificador aninhado cai com uma requisição de poucos quilobytes; em teste autorizado, `grep -rnE '\(\[?[^)]*[+*]\)?[+*]' --include='*.js' --include='*.py'` levanta candidatos. Isso é **análise de código com autorização do dono** — disparar a carga contra serviço de terceiro sem contrato é indisponibilidade provocada, enquadrada na **Lei 12.737/2012**. **Localidade muda a varredura:** `LC_ALL=C` deixa a semântica previsível e acelera bastante o `grep`, mas quebra o casamento de acentos — não use em texto em português. Fora do mundo GNU (macOS, BSD) não há `-P`, nem `\<`, nem `\K`: script portátil usa ERE puro.